

HookSuite

Memoria técnica · Práctica 3 — «Del prototipo al producto»

Máster en Ciberseguridad & IA · Evolve Academy

 **HUECO** — responsable: **José María**. Portada: nº de grupo definitivo, integrantes, fecha de entrega, enlace al repo y enlace al vídeo (apartado 1).

1. Portada

Producto: **HookSuite** — herramienta web de auditoría de seguridad «tipo Burp Suite», con IA. Grupo: **nº por confirmar**. Entrega: **16 de octubre de 2026**.

⚠ **HUECO** — responsable: **José María**. Integrantes con rol, enlace al repositorio público y enlace al vídeo (≥10 min). Se rellena al congelar.

2. Resumen ejecutivo

HookSuite resuelve la auditoría web asistida por IA desde el navegador: interceptar y repetir peticiones, fuzzing, utilidades (hash/encoder/regex), análisis de vulnerabilidades y un módulo de IA que prioriza hallazgos. La P3 parte del prototipo de la P1 y lo lleva a un estado ejecutable, seguro y reproducible.

⚠ **HUECO** — responsable: **José María + Claude**. Cerrar a 1 página cuando el producto esté congelado: qué hace, para quién, y en qué estado final se entrega.

3. Punto de partida

HookSuite nació en la Práctica 1 como prototipo funcional de una suite de auditoría web «tipo Burp Suite» operada desde el navegador y asistida por IA. Quedó desplegado en un servidor Hetzner (www.hooksuite.de) con Docker Compose y, desde la entrega de mayo de 2026, **nadie volvió a tocarlo**: el máster encadenó otras actividades y la Práctica 3 lo retoma tal como quedó.

Qué nos encontramos (verificación en vivo, 29-sep y 4-oct)

- La herramienta **seguía en pie** desde mayo, sin cambios: DNS → Hetzner, nginx/1.31.0, uvicorn y los 36 endpoints de la API respondiendo.
- Núcleo funcional sólido: Spider, Repeater, Intruder, Utilidades (hash/encoder/regex), sesiones por token UUID y WebSocket operativos.
- Dos hallazgos que **reorientan** la Práctica 3 (detalle en el apartado 7):
- La API :8000 estaba **abierta a internet sin ninguna autenticación**, incluidos los endpoints que lanzan tráfico contra terceros.
- En el historial de git había claves de API reales (secretos expuestos).
- El módulo de IA (RF-08), que es lo que da sentido al «con IA» del producto, **nunca llegó a operar** salvo pruebas muy iniciales de la P1: panel construido pero inactivo.

Numeración de requisitos

La P1 **no numeraba** los requisitos. El enunciado de la P3 (apartado 3) obliga a numerarlos ahora (RF-xx / RNF-xx) y a usar esa numeración en toda la práctica. La lista de abajo (apartado 4) está **derivada del informe de la P1** —de lo que el producto prometía y hace— y **PENDIENTE DE VALIDAR POR EL GRUPO** en la reunión antes de darla por firme.

Clasificación inicial de requisitos

Usando las cinco categorías del enunciado. El recuento es provisional, a validar por el grupo junto con la numeración.

Estado	Qué significa	Requisitos
Cumplido	Implementado en la P1 y funciona; en la P3 se verifica con prueba y evidencia	RF-01, RF-03, RF-04, RF-05, RF-06, RF-07, RF-11 · RNF-01, RNF-02, RNF-03, RNF-05, RNF-08 (12)
Parcial	Existe algo, pero incompleto o con fallos; hay que terminarlo y probarlo	RF-08 · RNF-04, RNF-07 (3)
Pendiente	Definido, pero sin implementar; se hace desde cero	RF-09, RF-10, RF-12 · RNF-09 (4)

A revisar	El feedback o la experiencia indican que estaba mal planteado; se reformula con justificación	RF-02 · RNF-06 (2)
Descartado	Se decide no implementarlo (debe ser la excepción y justificarse)	— ninguno de momento

⚠ **HUECO** — responsable: **José María**. Feedback docente de la P1: la calificación y los comentarios del equipo evaluador NO constan en el material de la línea. El enunciado los pide expresamente en este apartado. Conseguirlos (José María / portavoz) y resumir aquí qué se nos señaló y cómo lo aborda la P3.

4. Requisitos

Lista completa y numerada. Numeración fijada en la P3 (la P1 no la tenía) a partir del informe de la P1;

PENDIENTE DE VALIDAR POR EL GRUPO. El estado es el de partida (apartado 3); se actualiza a medida que la P3 avanza.

Requisitos funcionales (RF)

ID	Requisito	Estado P1
RF-01	Dashboard web accesible desde cualquier navegador, sin instalar nada en el cliente	Cumplido
RF-02	El servidor ejecuta las peticiones HTTP en nombre del auditor (proxy del lado servidor)	A revisar
RF-03	Spider: rastreo automático del objetivo (BFS, scope al dominio, extrae formularios), velocidad configurable	Cumplido
RF-04	Auditoría autenticada: sesión httpx persistente compartida por Spider/Repeater/Intruder que hereda las cookies del login	Cumplido
RF-05	Repeater: modificar y reenviar cualquier petición; respuesta en 4 vistas (Raw/Pretty/Preview/Headers)	Cumplido
RF-06	Intruder: fuzzing de parámetros marcados con *; 4 tipos (SQLi, Blind SQLi, XSS, genérico); resultados en tiempo real	Cumplido
RF-07	Utilidades: Encoder/Decoder (Base64/URL/HTML/JWT), Hash (MD5/SHA1/SHA256/SHA512), Regex Tester, Payload Generator	Cumplido
RF-08	Análisis y clasificación de vulnerabilidades con IA (Claude) según OWASP → panel Vulnerabilidades	Parcial
RF-09	Captura de tráfico HTTP en tiempo real (DevTools / CDP) → panel Red con código de colores	Pendiente
RF-10	Ejecución de ataques con navegador real (Playwright)	Pendiente
RF-11	Soporte multi-usuario simultáneo mediante tokens de sesión UUID	Cumplido
RF-12	Registro y login individual con JWT + aislamiento de historial/auditorías por usuario	Pendiente (mejora)

Requisitos no funcionales (RNF)

ID	Requisito	Estado P1
RNF-01	Accesible desde navegador sin instalar software en el cliente	Cumplido
RNF-02	Actualización en tiempo real vía WebSocket (pings cada 30 s, eventos {type,payload})	Cumplido
RNF-03	Desplegado en servidor Hetzner con contenedores Docker	Cumplido
RNF-04	Código en GitHub con historial de commits que refleje el trabajo del grupo	Parcial
RNF-05	Estabilidad ante carga: límite de concurrencia para no saturar producción	Cumplido
RNF-06	Operación degradada cuando la IA no responde o no supera el umbral de confianza	A revisar
RNF-07	Seguridad del propio producto : secretos fuera del repo, control de acceso, validación de entradas, dependencias sin CVEs	Parcial
RNF-08	Firewall dinámico por sesión (abre un puerto por sesión, lo cierra al logout o tras 4 h)	Cumplido (a verificar)
RNF-09	TLS/HTTPS en los endpoints públicos (Let's Encrypt)	Pendiente

Requisitos modificados o descartados (con justificación)

El enunciado pide que todo requisito modificado o descartado lleve su versión original, la nueva y el motivo.

RF-02 — proxy del lado servidor (modificado)

- **Versión original (P1):** interceptación del tráfico del navegador mediante archivo de autoconfiguración de proxy (PAC) + WebSockets.
- **Versión nueva:** el servidor ejecuta las peticiones directamente con `httpx`, sin interponerse en el navegador del auditor.
- **Motivo:** en la P1, el modelo PAC quedó expuesto y se saturó con tráfico de bots. El pivote a `httpx` directo elimina esa superficie y simplifica el flujo. Es el caso de libro de «requisito reformulado con justificación».

RNF-06 — operación degradada (a revisar)

- **Original:** enunciado como principio en la P1, sin demostrar.
- **Nuevo:** se demostrará con un umbral sobre el campo `confianza` que devuelve el clasificador de IA: por debajo del umbral, el hallazgo se marca como no concluyente en vez de descartarse. Encaja con el criterio P3 «se comporta razonablemente ante errores».

RF-12 — login individual con JWT (mejora, no requisito P1)

- No era requisito de la P1 (venía del road map de mejoras). En la P3 cuenta como **mejora**: suma, pero no compensa un requisito original sin cumplir. Se abordará por su valor de seguridad (cierra la fuga de datos entre sesiones).

> **Descartados:** ninguno por ahora. El enunciado pide que sean la excepción; si alguno se descarta (candidato: RF-09/DevTools si aprieta el tiempo) se justificará aquí por escrito.

5. Arquitectura y decisiones técnicas

HookSuite se despliega como un conjunto de contenedores Docker coordinados por `docker-compose`, todos en una red interna `hooksuite-net`, con Nginx como única puerta de entrada. Arquitectura verificada fichero a fichero en la cocina local el 4-oct (`docker-compose.yml` + `infra/nginx.conf`).

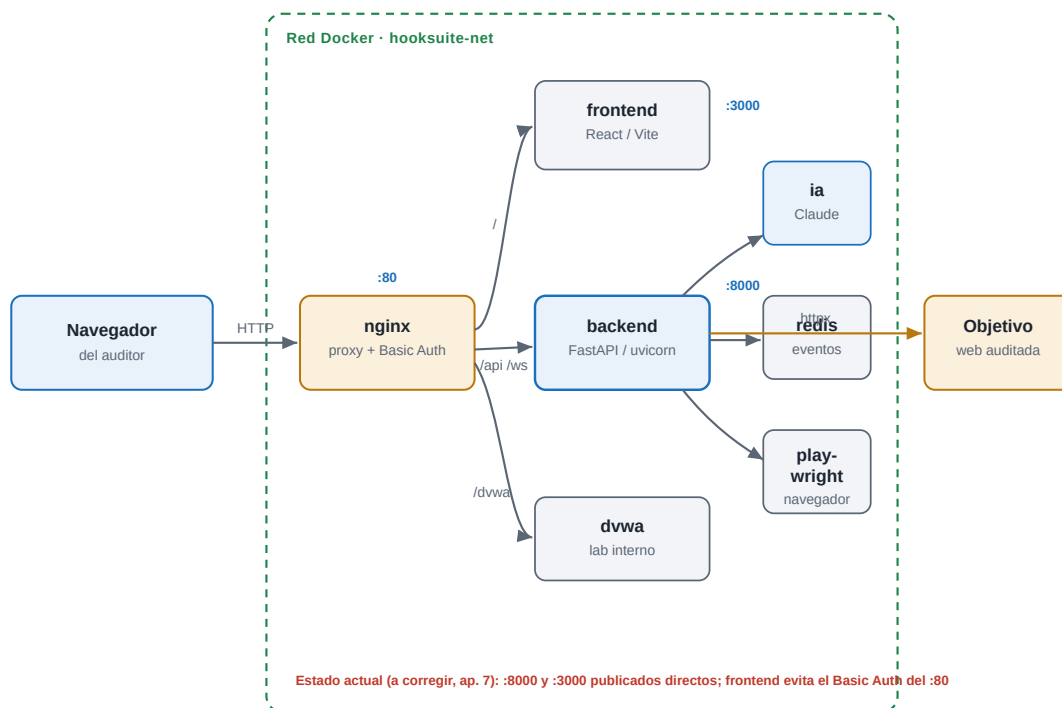


Figura 1: Arquitectura de contenedores de HookSuite: Nginx es la única entrada (Basic Auth); el backend FastAPI orquesta Spider/Repeater/Intruder/IA/Playwright y sale al objetivo auditado con `httpx`. En rojo, el estado actual a corregir (apartado 7).

Componentes

Componente	Tecnología	Rol	Puerto (prod.)
nginx	nginx:alpine	Proxy inverso y único punto de entrada; Basic Auth; enruta a frontend, backend y DVWA	80→80
frontend	React + Vite (build propio)	Dashboard web (SPA) servido como estático	3000→80
backend	FastAPI + uvicorn (build propio)	API de auditoría: proxy httpx, Spider, Repeater, Intruder, utilidades, vulnerabilidades y WebSocket	8000→8000
redis	redis:7-alpine	Bus de eventos y estado efímero entre servicios	interno
ia	Python + SDK Anthropic (build propio)	Clasificador de vulnerabilidades con Claude (RF-08)	interno
playwright	Python + Playwright (build propio)	Navegador real para ataques dirigidos (RF-10)	interno
dvwa	vulnerables/web-dvwa	Laboratorio de pruebas, solo accesible por la red interna	interno

Enrutado (Nginx)

Toda petición entra por Nginx, que reparte por ruta (`infra/nginx.conf`):

- `/` → **frontend** (protegido con Basic Auth).
- `/api/` y `/ws/` → **backend** (API y WebSocket).
- `/check/` y `/proxy.pac` → **backend** (utilidades del proxy).
- `/dvwa/` → **dvwa** (protegido con Basic Auth, solo interno).

Decisiones técnicas

Proxy del lado servidor: pivote PAC → httpx (RF-02)

La decisión arquitectónica de más peso heredada de la P1. En origen, HookSuite interceptaba el tráfico del navegador con un archivo de autoconfiguración (PAC) + WebSockets; ese modelo quedó expuesto y se saturó con tráfico de bots. Se pivotó a que **el servidor ejecute las peticiones directamente con httpx**: menos superficie y un flujo más simple, a cambio de perder la captura pasiva del tráfico del navegador —justo lo que retoman después DevTools (RF-09) y Playwright (RF-10)—. Versión original/nueva/justificación detalladas en el apartado 4.

Frontend: reescritura a Tailwind (cambio respecto a la P1)

La interfaz que corría en producción (`www.hooksuite.de`) se había reescrito al framework de estilos **Tailwind**, abandonando el sistema de variables CSS propio de la P1. Esa versión —la desplegada y en uso— es la que se toma como **base del frontend en la P3**. En la Fase 0 se reconcilió al repositorio (vivía solo en la caja, sin commitear); el detalle del proceso está en el apartado 8.

Puntos a corregir, detectados al montar la cocina

- **CORS abierto a cualquier origen** en el backend (FastAPI) → restringir a los orígenes propios (apartado 7).
- **Verificación SSL desactivada** en el cliente httpx: justificable en una herramienta de auditoría (objetivos con certificados autofirmados), pero se documenta expresamente como decisión, no como descuido.
- **Acoplamiento Nginx ↔ DVWA**: `infra/nginx.conf` declara `upstream dvwa`, así que **Nginx no arranca si DVWA no está** (`host not found in upstream`). El laboratorio es, por diseño, requisito de arranque del proxy —a desacoplar o justificar.
- **Playwright fuera de la red**: en el `docker-compose.yml` el servicio `playwright` no declara `networks: hooksuite-net`, así que queda aislado y no habla con el backend (causa raíz de RF-10/RF-08 sin operar; se corrige con una línea).

> **Nota de reproducibilidad (cocina)**. En el mediaserver los puertos `80/3000/8000` ya los ocupan otros servicios, así que la cocina los remapea a `8880/8830/8800` mediante un override local que no toca el `compose` versionado. Es solo del entorno de pruebas; la arquitectura de producción es la de la tabla.

⚠ **HUECO** — responsable: **José María + Claude**. Añadir, si procede, un diagrama de secuencia del ciclo IA ↔ Playwright ↔ Backend cuando RF-08/RF-10 estén operativos.

6. Funcionalidades implementadas

Cada funcionalidad con capturas: qué hace, cómo se usa y qué requisitos cubre.

- Proxy interceptor, Repeater, Intruder, Utilidades (hash/encoder/regex), Vulnerabilidades, Red/DevTools, módulo IA.

⚠ HUECO — responsable: **José María + Claude**. Documentar cada funcionalidad (qué hace, cómo se usa, qué requisitos cubre) con capturas de producto; las de pantalla las saca José María (R6), tras la Fase 3, cuando el login nuevo cambie las pantallas. El commit de cada módulo se firma a nombre de su responsable cuando toque (R2): Ivan/frontend, Macarena/backend, Nacho/playwright, Carlos/DevTools, José María/IA.

7. Seguridad del producto

Modelo de amenazas, secretos, autenticación, validación de entradas, dependencias y datos personales. (Base en REQUISITOS.md §3.) Contenido ya verificado en esta práctica:

Secretos: el incidente de las claves

En el historial de git había claves de API de Anthropic. Verificado el 4-oct: **las cuatro están muertas** (la API responde 401 / "API key is invalid."). Aun así, el enunciado (§7) obliga a limpiarlas del historial, no basta con borrarlas en un commit nuevo.

- El 4-oct se **reescribió el historial** con `git-filter-repo`: se redactaron las claves por patrón y se eliminaron los `.env` rastreados y la basura (`node_modules`, `venv` de Windows), conservando el código, las 7 ramas y el `.env.example` (con la clave ya redactada).
- **Ejecutado en GitHub el 4-oct** (`force-push` de las 7 ramas, con copia de seguridad previa). Verificado sobre un clon nuevo: **0 claves en todo el historial** de las ramas. El repositorio público queda sin secretos.
- Salvedad: GitHub conserva un tiempo los refs internos de los **pull requests** antiguos apuntando a los commits viejos; los purga por su cuenta. Como las claves están muertas, no supone riesgo.

LIMPIEZA EJECUTADA Y VERIFICADA

Exposición de la caja (antes de endurecer)

- La API `:8000` **no tiene autenticación**: lo declara su propia `openapi.json` (40 operaciones sin seguridad) y `/health` responde sin credenciales.
- El Basic Auth del `:80` se **evita** sirviendo el frontend directamente por `:3000`.
- DVWA **no** está expuesto a internet (solo red interna) — corrige un supuesto previo.
- Servidor: `nginx/1.31.0`.

A CORREGIR EN FASE 1

⚠ HUECO — responsable: **José María**. Completar el modelo STRIDE, validación de entradas, dependencias y tratamiento de datos personales.

8. Pruebas y evidencias

Plan de pruebas, casos ejecutados, resultados y las pruebas que fallaron con su explicación. El detalle caso a caso (PR-01...PR-20) se mantiene en la matriz del apartado 9 y en el diario del repo.

Evidencia de exposición (RNF-07 / PR-16), capturada el 4-oct

La prueba PR-16 —llamar a la API sin credenciales— y la verificación del frontend confirman que, antes de endurecer, el producto quedaba accesible sin autenticación. Capturas obtenidas antes de cualquier arreglo (R6: la evidencia de un fallo se captura antes de corregirlo, porque el propio arreglo la destruye):

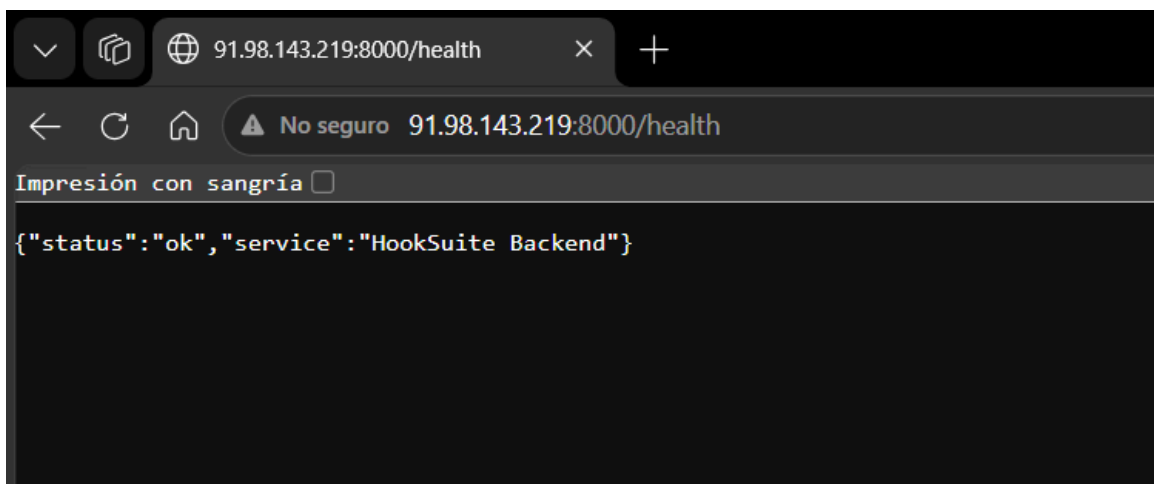


Figura 2: RNF-07 / PR-16 — GET /health de la API responde 200 sin credenciales ni cabecera WWW-Authenticate: la API no exige autenticación.

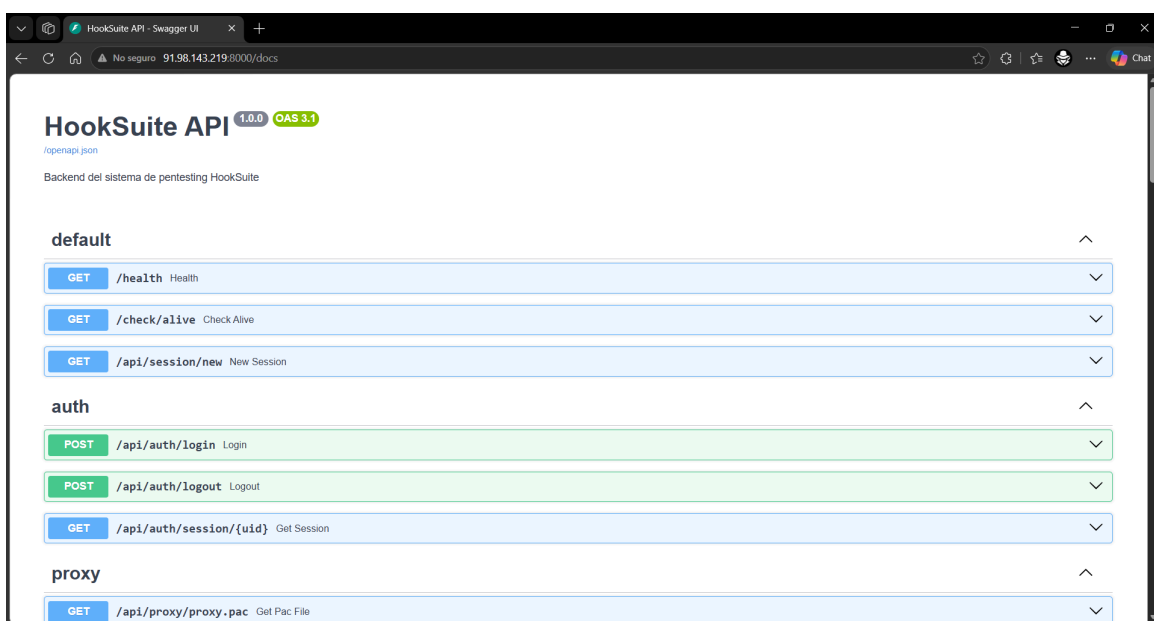


Figura 3: RNF-07 / PR-16 — la documentación Swagger (/docs) queda abierta a cualquiera, exponiendo las 40 operaciones de la API, incluidas las que lanzan tráfico contra terceros.

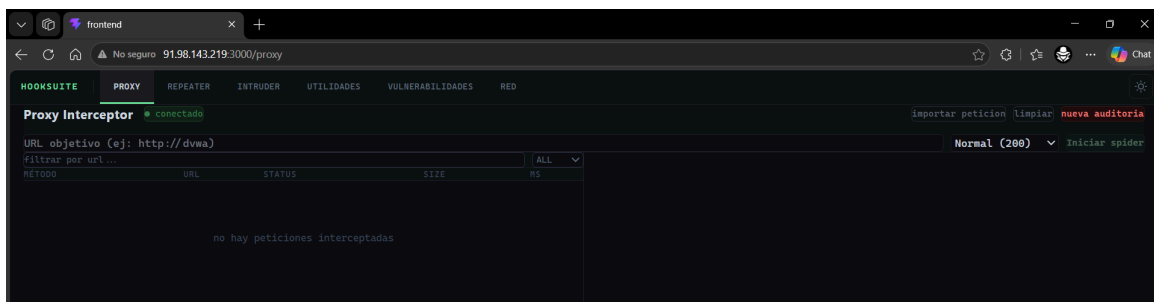


Figura 4: RNF-07 — el frontend servido directamente por :3000 carga 200 sin Basic Auth, evitando por completo el login de Nginx del :80.

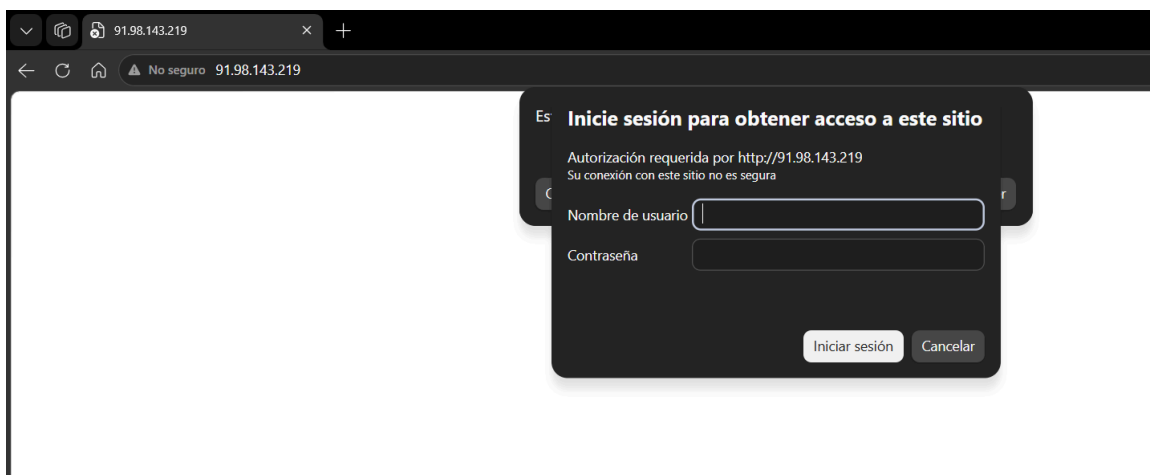


Figura 5: RNF-07 — por contraste, la entrada «correcta» (:80 tras Nginx) sí pide Basic Auth: el control existe, pero se puede rodear por el puerto directo.



Figura 6: RNF-07 — la credencial documentada en el informe de la P1 es rechazada con 401 por nginx/1.31.0: la credencial publicada ya no sirve, lo que refuerza el cambio de modelo de acceso en la Fase 2.

Otras evidencias ya en mano

- **Incidente de las claves y limpieza del historial:** ensayo de reescritura verificado antes/después (3 → 0 claves), detallado en el apartado 7.
- **Salidas de terminal** (health sin login, openapi.json sin seguridad) guardadas como texto en el repo, no como foto (R7: así ninguna captura publica por error un secreto).

EVIDENCIA DE EXPOSICIÓN COMPLETA

⚠ **HUECO** — responsable: **José María + Claude**. Plan de pruebas formal y capturas de PRODUCTO de los bugs (500 de `check/alive`, trampa del Spider) reproducidos en la cocina. Las de pantalla las saca José María (R6), tras la Fase 3, cuando el login nuevo cambie las pantallas. Las pruebas las hacen José María y Claude (R2).

9. Matriz de trazabilidad

La pieza con la que se corrige la práctica: requisito por requisito, dónde está implementado, qué prueba lo verifica y dónde se ve. Las rutas están comprobadas en el código (cocina local, 4-oct). La columna de evidencia enlaza la figura de la memoria y el **minuto exacto del vídeo**; se rellena al grabar.

Req.	Descripción	Estado	Implementación (ruta)	Prueba	Evidencia
RF-01	Dashboard web sin instalación	Cumplido	frontend/src/App.jsx + Layout.jsx	PR-01	fig. — · vídeo —:—
RF-02	Proxy del lado servidor (httpx)	Modificado	backend/services/proxy_service.py	PR-19	justif. apdo. 4
RF-03	Spider / rastreo	Cumplido	backend/services/spider_service.py	PR-03	fig. — · vídeo —:—
RF-04	Auditoría autenticada (sesión compartida)	Cumplido	backend/services/session_service.py	PR-04	fig. — · vídeo —:—

RF-05	Repeater	Cumplido	backend/routes/repeater.py	PR-05	fig. — · vídeo —:—
RF-06	Intruder / fuzzing	Cumplido	backend/services/intruder_service.py	PR-06	fig. — · vídeo —:—
RF-07	Utilidades (incl. JWT)	Cumplido	frontend/src/pages/utilities/	PR-07	fig. — · vídeo —:—
RF-08	IA → panel Vulnerabilidades	según alcance	ia/analyzers/vulnerability_classifier.py + backend/routes/vulnerabilities.py	PR-08	fig. — · vídeo —:—
RF-09	DevTools → panel Red	según alcance	devtools/ + backend/routes/network.py	PR-09	fig. — · vídeo —:—
RF-10	Playwright	según alcance	playwright/ + backend/routes/playwright.py	PR-10	fig. — · vídeo —:—
RF-11	Multi-usuario (tokens UUID)	Cumplido	backend/main.py (/api/session/new)	PR-11	fig. — · vídeo —:—
RF-12	Login JWT + aislamiento	mejora opcional	(por implementar, Fase 2)	PR-12	fig. — · vídeo —:—
RNF-02	WebSocket en tiempo real	Cumplido	backend/main.py (/ws/{token})	PR-17	vídeo —:—
RNF-03	Despliegue Docker en Hetzner	Cumplido	docker-compose.yml + infra/nginx.conf	PR-13	fig. — · vídeo —:—
RNF-05	Límite de concurrencia	Cumplido	backend/services/intruder_service.py (asyncio.Semaphore)	PR-20	apdo. 8
RNF-07	Seguridad del producto	a demostrar	ver apartado 7	PR-14, PR-16	apdo. 7-8
RNF-08	Firewall dinámico por sesión	verificar	firewall_agent.py	PR-15	apdo. 8
RNF-09	TLS/HTTPS	Pendiente	(por configurar en Nginx)	PR-18	apdo. 8

RUTAS VERIFICADAS EN CÓDIGO · PRUEBAS Y MINUTOS DE VÍDEO POR COMPLETAR

El estado de cada prueba (PR-xx) y su resultado detallado están en el apartado 8. Al arreglar cada fallo se reejecuta la prueba y se actualiza la fila. La numeración RF/RNF es la fijada en el apartado 4 y

PENDIENTE DE VALIDAR POR EL GRUPO

⚠ HUECO — responsable: **José María + Claude**. Al revisar el vídeo grabado, anotar el minuto exacto donde se demuestra cada requisito y el número de figura de su captura (R6: el nombre de la captura va atado al ID del requisito, p. ej. `RF-08-panel-ia.png`, para que esta columna se rellene sola).

10. Limitaciones y trabajo futuro

Qué no funciona, qué funciona con condiciones y qué se haría con más tiempo. Este apartado recibe material hasta el último día.

El estado de sesión es volátil, por diseño

El estado de una sesión de auditoría —tráfico capturado, resultados del Intruder, vulnerabilidades— vive **en memoria del backend**: al reiniciarlo, se pierde. Es una **decisión consciente**, no un olvido, y se sostiene en tres razones:

1. **El patrón de acceso del backend no lo soporta sin reescribirlo.** Una treintena de puntos obtienen el diccionario de la sesión y lo mutan en memoria (`session["requests"].append(...)`). Persistir de verdad obligaría a escritura-a-través en cada mutación, es decir, a tocar todo el backend a pocos días de la entrega.
2. **El modelo de sesión cambia en la Fase 2.** El login por usuario ata las sesiones a identidades; lo que se persistiera con el modelo anterior habría que rehacerlo.
3. **Una sesión de auditoría es un espacio de trabajo efímero.** El operador lanza un rastreo, examina los resultados y exporta lo que importa; no espera que la herramienta recuerde una sesión de hace tres días.

Lo que **sí** se hizo, porque era el riesgo real detrás de «todo en memoria»: **poner topes**. El recolector de sesiones existía en el código pero nunca se invocaba, y cualquier llamada a la API con un identificador nuevo creaba una sesión más —con la API sin autenticación, eso permitía agotar la memoria del servidor desde fuera—. Ahora hay un tope de sesiones y de peticiones por sesión (configurables por entorno), el recolector se ejecuta de verdad y, al desalojar, **prefiere las sesiones que no tienen conexión en tiempo real abierta**, para no interrumpir a quien está trabajando. Verificado con prueba de abuso: 120 identificadores inventados dejan el número de sesiones en el tope, y la sesión con conexión viva sobrevive.

Trabajo futuro. La persistencia es viable sin cambiar la arquitectura: el despliegue ya incluye Redis y el backend ya lo usa para otro flujo. El momento correcto es **después** del modelo de usuarios, para persistir sesiones ya asociadas a su propietario.

 **HUECO** — responsable: **José María + Claude**. Acumular el resto de limitaciones conforme aparezcan durante las Fases 1-4; este apartado recibe material hasta el último día.

11. Reparto del trabajo

Quién hizo qué, con estimación de horas por persona.

 **HUECO** — responsable: **José María**. Consolidar las horas por persona al congelar (13-oct). El reparto real del trabajo queda trazado por los commits firmados por cada responsable (R2); José María cierra el recuento.

12. Uso de herramientas de IA

Declaración obligatoria de qué herramientas de IA se usaron y para qué.

- **Claude Code** (modelo Claude de Anthropic): asistencia en diagnóstico, montaje de la cocina, ensayo de limpieza del historial y redacción de esta memoria, bajo revisión humana.

 **HUECO** — responsable: **José María**. Detallar por fase qué aportó la IA y qué fue decisión/revisión humana; distinguir generado de verificado.

13. Anexos

Manual de instalación ampliado, credenciales de prueba (ficticias), glosario y referencias.

 **HUECO** — responsable: **José María**. Manual de despliegue, usuarios de prueba del login nuevo (sustituyen a la credencial de Nginx), glosario y referencias.